Instituicao de Ensino Demo

Curso: Engenharia de Software

6o Semestre

Material de Estudo

Microsservicos e APIs

---

1. INTRODUCAO

Microsservicos e APIs sao a base da arquitetura de software moderna. Em vez de um unico sistema monolitico, aplicacoes sao decompostas em servicos pequenos e independentes que se comunicam via APIs. Isso permite que equipes trabalhem de forma autonoma, escalem servicos individualmente e usem tecnologias diferentes para cada servico. APIs RESTful e GraphQL sao os padroes dominantes para comunicacao entre sistemas na web.

Ementa: Arquitetura de microsservicos. Design de APIs RESTful. GraphQL. Comunicacao sincrona e assincrona. API Gateway. Service mesh. Documentacao com OpenAPI/Swagger. Versionamento e evolucao de APIs.

2. CONCEITOS FUNDAMENTAIS

A seguir, apresentamos os conceitos-chave desta disciplina com explicacoes didaticas e exemplos praticos para facilitar o aprendizado.

2.1 APIs RESTful e Boas Praticas

REST (Representational State Transfer) e o estilo arquitetural mais usado para APIs web. Recursos sao identificados por URLs, operacoes usam verbos HTTP (GET, POST, PUT, DELETE) e respostas usam status codes padrao (200 OK, 201 Created, 404 Not Found, 500 Error). Boas praticas incluem: nomes de recursos no plural, versionamento na URL, paginacao, HATEOAS e documentacao com OpenAPI.

Exemplo:

```
API RESTful para gerenciar alunos:
GET /api/v1/alunos -> lista alunos
GET /api/v1/alunos/123 -> detalhe do aluno 123
POST /api/v1/alunos -> cria novo aluno
PUT /api/v1/alunos/123 -> atualiza aluno 123
DELETE /api/v1/alunos/123 -> remove aluno 123

Resposta JSON:
{"id": 123, "nome": "Maria", "curso": "Eng. Software"}
```

2.2 Padroes de Microsservicos

Microsservicos trazem desafios que exigem padroes especificos. API Gateway centraliza o acesso externo. Circuit Breaker evita cascata de falhas. Saga gerencia transacoes distribuidas. Event Sourcing registra mudancas como eventos. CQRS separa leitura e escrita. Service Discovery permite que servicos se encontrem dinamicamente.

Exemplo:

```
Circuit Breaker (3 estados):
FECHADO: requisicoes passam normalmente
-> se falhas > limite: abre o circuito
ABERTO: requisicoes falham imediatamente (fast fail)
-> apos timeout: vai para meio-aberto
MEIO-ABERTO: permite uma requisicao de teste
-> se sucesso: fecha o circuito
-> se falha: abre novamente
```

3. RESUMO DOS PRINCIPAIS TOPICOS

O conteudo programatico da disciplina esta organizado nas seguintes unidades:

- Unidade 1: Monolitos vs microsservicos - vantagens e desafios
- Unidade 2: Design de APIs RESTful - recursos, verbos HTTP, status codes
- Unidade 3: GraphQL - queries, mutations, subscriptions
- Unidade 4: Comunicacao entre servicos - REST, gRPC, mensageria (RabbitMQ, Kafka)
- Unidade 5: API Gateway e service mesh (Istio, Envoy)
- Unidade 6: Autenticacao em APIs - JWT, OAuth 2.0, API Keys
- Unidade 7: Documentacao com OpenAPI/Swagger e versionamento
- Unidade 8: Padroes de resiliencia - Circuit Breaker, Retry, Bulkhead

Cada unidade aborda um aspecto fundamental da disciplina, construindo progressivamente o conhecimento necessario para dominar o tema. Recomenda-se estudar as unidades na ordem apresentada.

4. EXERCICIOS DE FIXACAO

Resolva os exercicios abaixo para verificar seu entendimento dos conceitos apresentados.

Questao 1 (Multipla Escolha)

Qual verbo HTTP e usado para criar um novo recurso em uma API REST?

a) GET
b) PUT
c) POST
d) PATCH

Questao 2 (Dissertativa)

Compare REST e GraphQL: vantagens e desvantagens de cada abordagem.

Questao 3 (Multipla Escolha)

Qual padrao evita cascata de falhas entre microsservicos?

a) Singleton
b) Circuit Breaker
c) Observer
d) Factory

Questao 4 (Dissertativa)

Projete uma API RESTful para um sistema de e-commerce com recursos: produtos, pedidos e clientes.

5. DICAS DE ESTUDO E RECURSOS ADICIONAIS

Para maximizar seu aprendizado nesta disciplina, siga estas recomendacoes:

- Construa APIs RESTful com Flask ou FastAPI (Python) para praticar.
- Estude autenticacao com JWT e OAuth 2.0 na pratica.
- Use Postman ou Insomnia para testar suas APIs.
- Leia o livro "Construindo Microsservicos" de Sam Newman.
- Documente suas APIs com Swagger/OpenAPI desde o inicio.

Bibliografia Recomendada:

- NEWMAN, S. Construindo Microsservicos. Novatec, 2022.
- RICHARDSON, C. Microsservicos: Padroes e Aplicacoes. Alta Books, 2019.
- MASSE, M. REST API Design Rulebook. O'Reilly, 2011.

6. GLOSSARIO DE TERMOS IMPORTANTES

Consulte este glossario sempre que encontrar um termo desconhecido durante seus estudos.

- **API**: Application Programming Interface — interface de comunicacao entre sistemas.
- **REST**: Representational State Transfer — estilo arquitetural para APIs web.
- **GraphQL**: linguagem de consulta para APIs desenvolvida pelo Facebook.
- **Microsservico**: servico pequeno e independente com responsabilidade unica.
- **API Gateway**: ponto de entrada unico para todas as requisicoes externas.
- **Circuit Breaker**: padrao que evita cascata de falhas entre servicos.
- **JWT**: JSON Web Token — padrao para autenticacao stateless.
- **gRPC**: framework de RPC de alta performance do Google.